

Cascading rollback is undesirable, since it leads to the undoing of a significant amount of work. It is desirable to restrict the schedules to those where cascading rollbacks cannot occur. Such schedules are called *cascadeless* schedules. Formally, a **cascadeless schedule** is one where, for each pair of transactions T_i and T_j such that T_j reads a data item previously written by T_i , the commit operation of T_i appears before the read operation of T_j . It is easy to verify that every cascadeless schedule is also recoverable.

17.8 Transaction Isolation Levels

Serializability is a useful concept because it allows programmers to ignore issues related to concurrency when they code transactions. If every transaction has the property that it maintains database consistency if executed alone, then serializability ensures that concurrent executions maintain consistency. However, the protocols required to ensure serializability may allow too little concurrency for certain applications. In these cases, weaker levels of consistency are used. The use of weaker levels of consistency places additional burdens on programmers for ensuring database correctness.

The SQL standard also allows a transaction to specify that it may be executed in such a way that it becomes nonserializable with respect to other transactions. For instance, a transaction may operate at the isolation level of **read uncommitted**, which permits the transaction to read a data item even if it was written by a transaction that has not been committed. SQL provides such features for the benefit of long transactions whose results do not need to be precise. If these transactions were to execute in a serializable fashion, they could interfere with other transactions, causing the others' execution to be delayed.

The **isolation levels** specified by the SQL standard are as follows:

- **Serializable** usually ensures serializable execution. However, as we shall explain shortly, some database systems implement this isolation level in a manner that may, in certain cases, allow nonserializable executions.
- **Repeatable read** allows only committed data to be read and further requires that, between two reads of a data item by a transaction, no other transaction is allowed to update it. However, the transaction may not be serializable with respect to other transactions. For instance, when it is searching for data satisfying some conditions, a transaction may find some of the data inserted by a committed transaction, but may not find other data inserted by the same transaction.
- **Read committed** allows only committed data to be read, but does not require repeatable reads. For instance, between two reads of a data item by the transaction, another transaction may have updated the data item and committed.
- **Read uncommitted** allows uncommitted data to be read. It is the lowest isolation level allowed by SQL.

All the isolation levels above additionally disallow **dirty writes**, that is, they disallow writes to a data item that has already been written by another transaction that has not yet committed or aborted.

Many database systems run, by default, at the read-committed isolation level. In SQL, it is possible to set the isolation level explicitly, rather than accepting the system's default setting. For example, the statement

set transaction isolation level serializable

sets the isolation level to serializable; any of the other isolation levels may be specified instead. The preceding syntax is supported by Oracle, PostgreSQL, and SQL Server; Oracle uses the syntax

alter session set isolation_level = serializable

while DB2 uses the syntax “**change isolation level**” with its own abbreviations for isolation levels. Changing of the isolation level must be done as the first statement of a transaction.

By default, most databases commit individual statements as soon as they are executed. Such **automatic commit** of individual statements must be turned off to allow multiple statements to run as a single transaction. The command **start transaction** ensures that subsequent SQL statements, until a subsequent **commit** or **rollback**, are executed as a single transaction. As expected, the **commit** operation commits the preceding SQL statements, while **rollback** rolls back the preceding SQL statements. (SQL Server uses **begin transaction** in place of **start transaction**, while Oracle and PostgreSQL treat **begin** as identical to **start transaction**.)

APIs such as JDBC and ODBC provide functions to turn off automatic commit. In JDBC the `setAutoCommit` method of the `Connection` interface (which we saw earlier in Section 5.1.1.8) can be used to turn automatic commit off by invoking `setAutoCommit(false)`, or on by invoking `setAutoCommit(true)`. Further, in JDBC the method `setTransactionIsolation(int level)` of the `Connection` interface can be invoked with any one of

- `Connection.TRANSACTION_SERIALIZABLE`,
- `Connection.TRANSACTION_REPEATABLE_READ`,
- `Connection.TRANSACTION_READ_COMMITTED`, or
- `Connection.TRANSACTION_READ_UNCOMMITTED`

to set the transaction isolation level correspondingly.

An application designer may decide to accept a weaker isolation level in order to improve system performance. As we shall see in Section 17.9 and Chapter 18, ensuring serializability may force a transaction to wait for other transactions or, in some cases, to abort because the transaction can no longer be executed as part of a serializable execution. While it may seem shortsighted to risk database consistency for performance,

this trade-off makes sense if we can be sure that the inconsistency that may occur is not relevant to the application.

There are many means of implementing isolation levels. As long as the implementation ensures serializability, the designer of a database application or a user of an application does not need to know the details of such implementations, except perhaps for dealing with performance issues. Unfortunately, even if the isolation level is set to **serializable**, some database systems actually implement a weaker level of isolation, which does not rule out every possible nonserializable execution; we revisit this issue in Section 17.9. If weaker levels of isolation are used, either explicitly or implicitly, the application designer has to be aware of some details of the implementation, to avoid or minimize the chance of inconsistency due to lack of serializability.

17.9 Implementation of Isolation Levels

So far, we have seen what properties a schedule must have if it is to leave the database in a consistent state and allow transaction failures to be handled in a safe manner.

There are various **concurrency-control** policies that we can use to ensure that, even when multiple transactions are executed concurrently, only acceptable schedules are generated, regardless of how the operating system time-shares resources (such as CPU time) among the transactions.

As a trivial example of a concurrency-control policy, consider this: A transaction acquires a **lock** on the entire database before it starts and releases the lock after it has committed. While a transaction holds a lock, no other transaction is allowed to acquire the lock, and all must therefore wait for the lock to be released. As a result of the locking policy, only one transaction can execute at a time. Therefore, only serial schedules are generated. These are trivially serializable, and it is easy to verify that they are recoverable and cascadeless as well.

A concurrency-control policy such as this one leads to poor performance, since it forces transactions to wait for preceding transactions to finish before they can start. In other words, it provides a poor degree of concurrency (indeed, no concurrency at all). As we saw in Section 17.5, concurrent execution has substantial performance benefits.

The goal of concurrency-control policies is to provide a high degree of concurrency, while ensuring that all schedules that can be generated are conflict or view serializable, recoverable, and cascadeless.

Here we provide an overview of how some of most important concurrency-control mechanisms work, and we defer the details to Chapter 18.

17.9.1 Locking

Instead of locking the entire database, a transaction could instead lock only those data items that it accesses. Under such a policy, the transaction must hold locks long enough to ensure serializability, but for a period short enough not to harm performance exces-

Note 17.2 SERIALIZABILITY IN THE REAL WORLD

Serializable schedules are the ideal way to ensure consistency, but in our day-to-day lives, we don't impose such stringent requirements. A web site offering goods for sale may list an item as being in stock, yet by the time a user selects the item and goes through the checkout process, that item might no longer be available. Viewed from a database perspective, this would be a nonrepeatable read.

As another example, consider seat selection for air travel. Assume that a traveler has already booked an itinerary and now is selecting seats for each flight. Many airline web sites allow the user to step through the various flights and choose a seat, after which the user is asked to confirm the selection. It could be that other travelers are selecting seats or changing their seat selections for the same flights at the same time. The seat availability that the traveler was shown is thus actually changing, but the traveler is shown a snapshot of the seat availability as of when the traveler started the seat selection process.

Even if two travelers are selecting seats at the same time, most likely they will select different seats, and if so there would be no real conflict. However, the transactions are not serializable, since each traveler has read data that was subsequently updated by the other traveler, leading to a cycle in the precedence graph. If two travelers performing seat selection concurrently actually selected the same seat, one of them would not be able to get the seat they selected; however, the situation could be easily resolved by asking the traveler to perform the selection again, with updated seat availability information.

It is possible to enforce serializability by allowing only one traveler to do seat selection for a particular flight at a time. However, doing so could cause significant delays as travelers would have to wait for their flight to become available for seat selection; in particular a traveler who takes a long time to make a choice could cause serious problems for other travelers. Instead, any such transaction is typically broken up into a part that requires user interaction and a part that runs exclusively on the database. In the example above, the database transaction would check if the seats chosen by the user are still available, and if so update the seat selection in the database. Serializability is ensured only for the transactions that run on the database, without user interaction.

sively. Complicating matters are SQL statements where the data items accessed depend on a **where** clause, which we discuss in Section 17.10. In Chapter 18, we present the two-phase locking protocol, a simple, widely used technique that ensures serializability. Stated simply, two-phase locking requires a transaction to have two phases, one where it acquires locks but does not release any, and a second phase where the transaction releases locks but does not acquire any. (In practice, locks are usually released only when the transaction completes its execution and has been either committed or aborted.)

Further improvements to locking result if we have two kinds of locks: shared and exclusive. Shared locks are used for data that the transaction reads and exclusive locks are used for those it writes. Many transactions can hold shared locks on the same data item at the same time, but a transaction is allowed an exclusive lock on a data item only if no other transaction holds any lock (regardless of whether shared or exclusive) on the data item. This use of two modes of locks along with two-phase locking allows concurrent reading of data while still ensuring serializability.

17.9.2 Timestamps

Another category of techniques for the implementation of isolation assigns each transaction a **timestamp**, typically when it begins. For each data item, the system keeps two timestamps. The read timestamp of a data item holds the largest (that is, the most recent) timestamp of those transactions that read the data item. The write timestamp of a data item holds the timestamp of the transaction that wrote the current value of the data item. Timestamps are used to ensure that transactions access each data item in order of the transactions' timestamps if their accesses conflict. When this is not possible, offending transactions are aborted and restarted with a new timestamp.

17.9.3 Multiple Versions and Snapshot Isolation

By maintaining more than one version of a data item, it is possible to allow a transaction to read an old version of a data item rather than a newer version written by an uncommitted transaction or by a transaction that should come later in the serialization order. There are a variety of multiversion concurrency-control techniques. One in particular, called **snapshot isolation**, is widely used in practice.

In snapshot isolation, we can imagine that each transaction is given its own version, or snapshot, of the database when it begins.⁵ It reads data from this private version and is thus isolated from the updates made by other transactions. If the transaction updates the database, that update appears only in its own version, not in the actual database itself. Information about these updates is saved so that the updates can be applied to the “real” database if the transaction commits.

When a transaction T enters the partially committed state, it then proceeds to the committed state only if no other concurrent transaction has modified data that T intends to update. Transactions that, as a result, cannot commit abort instead.

Snapshot isolation ensures that attempts to read data never need to wait (unlike locking). Read-only transactions cannot be aborted; only those that modify data run a slight risk of aborting. Since each transaction reads its own version or snapshot of the database, reading data does not cause subsequent update attempts by other transactions to wait (unlike locking). Since most transactions are read-only (and most others read more data than they update), this is often a major source of performance improvement as compared to locking.

⁵In reality, the entire database is not copied. Multiple versions are kept only of those data items that are changed.

The number of locks that an SQL query may need to acquire can usually be estimated based on the relation scan operations performed by a query. A relation scan, for example, would acquire a lock at a relation level, while an index scan that is expected to fetch only a few records may acquire an intention lock at the relation level and regular locks at the tuple level. In case the a transaction acquires a large number of tuple locks, the lock table may become overfull. To deal with this situation, the lock manager may perform **lock escalation**, replacing many lower level locks by a single higher level lock; in our example, a single relation lock could replace a large number of tuple locks.

18.4 Insert Operations, Delete Operations, and Predicate Reads

Until now, we have restricted our attention to **read** and **write** operations. This restriction limits transactions to data items already in the database. Some transactions require not only access to existing data items, but also the ability to create new data items. Others require the ability to delete data items. To examine how such transactions affect concurrency control, we introduce these additional operations:

- **delete(Q)** deletes data item Q from the database.
- **insert(Q)** inserts a new data item Q into the database and assigns Q an initial value.

An attempt by a transaction T_i to perform a **read(Q)** operation after Q has been deleted results in a logical error in T_i . Likewise, an attempt by a transaction T_i to perform a **read(Q)** operation before Q has been inserted results in a logical error in T_i . It is also a logical error to attempt to delete a nonexistent data item.

18.4.1 Deletion

To understand how the presence of **delete** instructions affects concurrency control, we must decide when a **delete** instruction conflicts with another instruction. Let I_i and I_j be instructions of T_i and T_j , respectively, that appear in schedule S in consecutive order. Let $I_i = \mathbf{delete}(Q)$. We consider several instructions I_j .

- $I_j = \mathbf{read}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_j can execute the **read** operation successfully.
- $I_j = \mathbf{write}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_j can execute the **write** operation successfully.
- $I_j = \mathbf{delete}(Q)$. I_i and I_j conflict. If I_i comes before I_j , T_j will have a logical error. If I_j comes before I_i , T_i will have a logical error.
- $I_j = \mathbf{insert}(Q)$. I_i and I_j conflict. Suppose that data item Q did not exist prior to the execution of I_i and I_j . Then, if I_i comes before I_j , a logical error results for T_i .

If I_j comes before I_i , then no logical error results. Likewise, if Q existed prior to the execution of I_i and I_j , then a logical error results if I_j comes before I_i , but not otherwise.

We can conclude the following:

- Under the two-phase locking protocol, an exclusive lock is required on a data item before that item can be deleted.
- Under the timestamp-ordering protocol, a test similar to that for a **write** must be performed. Suppose that transaction T_i issues **delete**(Q).
 - If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i was to delete has already been read by a transaction T_j with $TS(T_j) > TS(T_i)$. Hence, the **delete** operation is rejected, and T_i is rolled back.
 - If $TS(T_i) < W\text{-timestamp}(Q)$, then a transaction T_j with $TS(T_j) > TS(T_i)$ has written Q . Hence, this **delete** operation is rejected, and T_i is rolled back.
 - Otherwise, the **delete** is executed.

18.4.2 Insertion

We have already seen that an **insert**(Q) operation conflicts with a **delete**(Q) operation. Similarly, **insert**(Q) conflicts with a **read**(Q) operation or a **write**(Q) operation; no read or write can be performed on a data item before it exists.

Since an **insert**(Q) assigns a value to data item Q , an **insert** is treated similarly to a **write** for concurrency-control purposes:

- Under the two-phase locking protocol, if T_i performs an **insert**(Q) operation, T_i is given an exclusive lock on the newly created data item Q .
- Under the timestamp-ordering protocol, if T_i performs an **insert**(Q) operation, the values $R\text{-timestamp}(Q)$ and $W\text{-timestamp}(Q)$ are set to $TS(T_i)$.

18.4.3 Predicate Reads and The Phantom Phenomenon

Consider transaction T_{30} that executes the following SQL query on the university database:

```
select count(*)
from instructor
where dept_name = 'Physics' ;
```

Transaction T_{30} requires access to all tuples of the *instructor* relation pertaining to the Physics department.

Let T_{31} be a transaction that executes the following SQL insertion:

```
insert into instructor
values (11111, 'Feynman', 'Physics', 94000);
```

Let S be a schedule involving T_{30} and T_{31} . We expect there to be potential for a conflict for the following reasons:

- If T_{30} uses the tuple newly inserted by T_{31} in computing **count**(*), then T_{30} reads a value written by T_{31} . Thus, in a serial schedule equivalent to S , T_{31} must come before T_{30} .
- If T_{30} does not use the tuple newly inserted by T_{31} in computing **count**(*), then in a serial schedule equivalent to S , T_{30} must come before T_{31} .

The second of these two cases is curious. T_{30} and T_{31} do not access any tuple in common, yet they conflict with each other! In effect, T_{30} and T_{31} conflict on a phantom tuple. If concurrency control is performed at the tuple granularity, this conflict would go undetected. As a result, the system could fail to prevent a nonserializable schedule. This problem is an instance of the **phantom phenomenon**.

Phantom phenomena can occur not just with inserts, but also with updates. Consider the situation we saw in Section 17.10, where a transaction T_i used an index to find only tuples with *dept_name* = “Physics”, and as a result did not read any tuples with other department names. If another transaction T_j updates one of these tuples, changing its department name to Physics, a problem similar to the above problem occurs: even though T_i and T_j have not accessed any tuples in common, they do conflict with each other. This problem too is an instance of the phantom phenomenon. In general, the phantom phenomenon is rooted in predicate reads that conflict with inserts or updates that result in new/updated tuples that satisfy the predicate.

We can prevent these problems by allowing transaction T_{30} to prevent other transactions from creating new tuples in the *instructor* relation with *dept_name* = “Physics”, and from updating the department name of an existing *instructor* tuple to Physics.

To find all *instructor* tuples with *dept_name* = “Physics”, T_{30} must search either the whole *instructor* relation, or at least an index on the relation. Up to now, we have assumed implicitly that the only data items accessed by a transaction are tuples. However, T_{30} is an example of a transaction that reads information about what tuples are in a relation, and T_{31} is an example of a transaction that updates that information.

Clearly, it is not sufficient merely to lock the tuples that are accessed; the information used to find the tuples that are accessed by the transaction must also be locked.

Locking of information used to find tuples can be implemented by associating a data item with the relation; the data item represents the information used to find the tuples in the relation. Transactions, such as T_{30} , that read the information about what tuples are in a relation would then have to lock the data item corresponding to the

relation in shared mode. Transactions, such as T_{31} , that update the information about what tuples are in a relation would have to lock the data item in exclusive mode. Thus, T_{30} and T_{31} would conflict on a real data item, rather than on a phantom. Similarly, transactions that use an index to retrieve tuples must lock the index itself.

Do not confuse the locking of an entire relation, as in multiple-granularity locking, with the locking of the data item corresponding to the relation. By locking the data item, a transaction only prevents other transactions from updating information about what tuples are in the relation. Locking is still required on tuples. A transaction that directly accesses a tuple can be granted a lock on the tuples even when another transaction has an exclusive lock on the data item corresponding to the relation itself.

The major disadvantage of locking a data item corresponding to the relation, or locking an entire index, is the low degree of concurrency—two transactions that insert different tuples into a relation are prevented from executing concurrently.

A better solution is an **index-locking** technique that avoids locking the whole index. Any transaction that inserts a tuple into a relation must insert information into every index maintained on the relation. We eliminate the phantom phenomenon by imposing a locking protocol for indices. For simplicity we shall consider only B⁺-tree indices.

As we saw in Chapter 14, every search-key value is associated with an index leaf node. A query will usually use one or more indices to access a relation. An insert must insert the new tuple in all indices on the relation. In our example, we assume that there is an index on *instructor* for attribute *dept_name*. Then, T_{31} must modify the leaf containing the key “Physics”. If T_{30} reads the same leaf node to locate all tuples pertaining to the Physics department, then T_{30} and T_{31} conflict on that leaf node.

The **index-locking protocol** takes advantage of the availability of indices on a relation, by turning instances of the phantom phenomenon into conflicts on locks on index leaf nodes. The protocol operates as follows:

- Every relation must have at least one index.
- A transaction T_i can access tuples of a relation only after first finding them through one or more of the indices on the relation. For the purpose of the index-locking protocol, a relation scan is treated as a scan through all the leaves of one of the indices.
- A transaction T_i that performs a lookup (whether a range lookup or a point lookup) must acquire a shared lock on all the index leaf nodes that it accesses.
- A transaction T_i may not insert, delete, or update a tuple t_i in a relation r without updating all indices on r . The transaction must obtain exclusive locks on all index leaf nodes that are affected by the insertion, deletion, or update. For insertion and deletion, the leaf nodes affected are those that contain (after insertion) or contained (before deletion) the search-key value of the tuple. For updates, the leaf nodes affected are those that (before the modification) contained the old value of the search key, and nodes that (after the modification) contain the new value of the search key.

- Locks are obtained on tuples as usual.
- The rules of the two-phase locking protocol must be observed.

Note that the index-locking protocol does not address concurrency control on internal nodes of an index; techniques for concurrency control on indices, which minimize lock conflicts, are presented in Section 18.10.2.

Locking an index leaf node prevents any update to the node, even if the update did not actually conflict with the predicate. A variant called key-value locking, which minimizes such false lock conflicts, is presented in Section 18.10.2 as part of index concurrency control.

As noted in Section 17.10, it would appear that the existence of a conflict between transactions depends on a low-level query-processing decision by the system that is unrelated to a user-level view of the meaning of the two transactions. An alternative approach to concurrency control acquires shared locks on predicates in a query, such as the predicate “*salary* > 90000” on the *instructor* relation. Inserts and deletes of the relation must then be checked to see if they satisfy the predicate; if they do, there is a lock conflict, forcing the insert or delete to wait till the predicate lock is released. For updates, both the initial value and the final value of the tuple must be checked against the predicate. Such conflicting inserts, deletes, and updates affect the set of tuples selected by the predicate, and they cannot be allowed to execute concurrently with the query that acquired the (shared) predicate lock. We call this protocol **predicate locking**;¹ predicate locking is not used in practice since it is more expensive to implement than the index-locking protocol and does not give significant additional benefits.

18.5 Timestamp-Based Protocols

The locking protocols that we have described thus far determine the order between every pair of conflicting transactions at execution time by the first lock that both members of the pair request that involves incompatible modes. Another method for determining the serializability order is to select an ordering among transactions in advance. The most common method for doing so is to use a *timestamp-ordering* scheme.

18.5.1 Timestamps

With each transaction T_i in the system, we associate a unique fixed timestamp, denoted by $TS(T_i)$. This timestamp is assigned by the database system before the transaction T_i starts execution. If a transaction T_i has been assigned timestamp $TS(T_i)$, and a new transaction T_j enters the system, then $TS(T_i) < TS(T_j)$. There are two simple methods for implementing this scheme:

¹The term *predicate locking* was used for a version of the protocol that used shared and exclusive locks on predicates, and was thus more complicated. The version we present here, with only shared locks on predicates, is also referred to as **precision locking**.

1. Use the value of the **system clock** as the timestamp; that is, a transaction's timestamp is equal to the value of the clock when the transaction enters the system.
2. Use a **logical counter** that is incremented after a new timestamp has been assigned; that is, a transaction's timestamp is equal to the value of the counter when the transaction enters the system.

The timestamps of the transactions determine the serializability order. Thus, if $TS(T_i) < TS(T_j)$, then the system must ensure that the produced schedule is equivalent to a serial schedule in which transaction T_i appears before transaction T_j .

To implement this scheme, we associate with each data item Q two timestamp values:

1. **W-timestamp**(Q) denotes the largest timestamp of any transaction that executed **write**(Q) successfully.
2. **R-timestamp**(Q) denotes the largest timestamp of any transaction that executed **read**(Q) successfully.

These timestamps are updated whenever a new **read**(Q) or **write**(Q) instruction is executed.

18.5.2 The Timestamp-Ordering Protocol

The **timestamp-ordering protocol** ensures that any conflicting read and write operations are executed in timestamp order. This protocol operates as follows:

- Suppose that transaction T_i issues **read**(Q).
 - If $TS(T_i) < \text{W-timestamp}(Q)$, then T_i needs to read a value of Q that was already overwritten. Hence, the **read** operation is rejected, and T_i is rolled back.
 - If $TS(T_i) \geq \text{W-timestamp}(Q)$, then the **read** operation is executed, and **R-timestamp**(Q) is set to the maximum of **R-timestamp**(Q) and $TS(T_i)$.
- Suppose that transaction T_i issues **write**(Q).
 - If $TS(T_i) < \text{R-timestamp}(Q)$, then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced. Hence, the system rejects the **write** operation and rolls T_i back.
 - If $TS(T_i) < \text{W-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, the system rejects this **write** operation and rolls T_i back.
 - Otherwise, the system executes the **write** operation and sets **W-timestamp**(Q) to $TS(T_i)$.

If a transaction T_i is rolled back by the concurrency-control scheme as result of issuance of either a read or write operation, the system assigns it a new timestamp and restarts it.

To illustrate this protocol, we consider transactions T_{25} and T_{26} . Transaction T_{25} displays the contents of accounts A and B :

```
 $T_{25}$ : read( $B$ );
      read( $A$ );
      display( $A + B$ ).
```

Transaction T_{26} transfers \$50 from account B to account A , and then displays the contents of both:

```
 $T_{26}$ : read( $B$ );
       $B := B - 50$ ;
      write( $B$ );
      read( $A$ );
       $A := A + 50$ ;
      write( $A$ );
      display( $A + B$ ).
```

In presenting schedules under the timestamp protocol, we shall assume that a transaction is assigned a timestamp immediately before its first instruction. Thus, in schedule 3 of Figure 18.17, $TS(T_{25}) < TS(T_{26})$, and the schedule is possible under the timestamp protocol.

We note that the preceding execution can also be produced by the two-phase locking protocol. There are, however, schedules that are possible under the two-phase locking protocol, but are not possible under the timestamp protocol, and vice versa (see Exercise 18.27).

The timestamp-ordering protocol ensures conflict serializability. This is because conflicting operations are processed in timestamp order.

The protocol ensures freedom from deadlock, since no transaction ever waits. However, there is a possibility of starvation of long transactions if a sequence of conflicting short transactions causes repeated restarting of the long transaction. If a transaction is suffering from repeated restarts, conflicting transactions need to be temporarily blocked to enable the transaction to finish.

The protocol can generate schedules that are not recoverable. However, it can be extended to make the schedules recoverable, in one of several ways:

- Recoverability and cascadelessness can be ensured by performing all writes together at the end of the transaction. The writes must be atomic in the following sense: While the writes are in progress, no transaction is permitted to access any of the data items that have been written.

T_{25}	T_{26}
read(B)	read(B) $B := B - 50$ write(B)
read(A)	read(A)
display($A + B$)	$A := A + 50$ write(A) display($A + B$)

Figure 18.17 Schedule 3.

- Recoverability and cascadelessness can also be guaranteed by using a limited form of locking, whereby reads of uncommitted items are postponed until the transaction that updated the item commits (see Exercise 18.28).
- Recoverability alone can be ensured by tracking uncommitted writes and allowing a transaction T_i to commit only after the commit of any transaction that wrote a value that T_i read. Commit dependencies, outlined in Section 18.1.5, can be used for this purpose.

If the timestamp-ordering protocol is applied only to tuples, the protocol would be vulnerable to the phantom problems that we saw in Section 17.10 and Section 18.4.3.

To avoid this problem, the timestamp-ordering protocol could be applied to all data that is read by a transaction, including relation metadata and index data. In the context of locking-based concurrency control, the index-locking protocol, described in Section 18.4.3, is a more efficient alternative for avoiding the phantom problem; recall that the index-locking protocol obtains locks on index nodes, in addition to obtaining locks on tuples. The timestamp-ordering protocol can be similarly modified to treat each index node as a data item, with associated read and write timestamps, and to apply the timestamp-ordering tests on these data items, too. This extended version of the timestamp-ordering protocol avoids phantom problems and ensures serializability even with predicate reads.

18.5.3 Thomas' Write Rule

We now present a modification to the timestamp-ordering protocol that allows greater potential concurrency than does the protocol of Section 18.5.2. Let us consider schedule 4 of Figure 18.18 and apply the timestamp-ordering protocol. Since T_{27} starts before T_{28} , we shall assume that $TS(T_{27}) < TS(T_{28})$. The $read(Q)$ operation of T_{27} succeeds, as does the $write(Q)$ operation of T_{28} . When T_{27} attempts its $write(Q)$ operation,

T_{27}	T_{28}
read(Q)	write(Q)
write(Q)	

Figure 18.18 Schedule 4.

we find that $TS(T_{27}) < W\text{-timestamp}(Q)$, since $W\text{-timestamp}(Q) = TS(T_{28})$. Thus, the `write( $Q$ )` by T_{27} is rejected and transaction T_{27} must be rolled back.

Although the rollback of T_{27} is required by the timestamp-ordering protocol, it is unnecessary. Since T_{28} has already written Q , the value that T_{27} is attempting to write is one that will never need to be read. Any transaction T_i with $TS(T_i) < TS(T_{28})$ that attempts a `read( $Q$ )` will be rolled back, since $TS(T_i) < W\text{-timestamp}(Q)$. Any transaction T_j with $TS(T_j) > TS(T_{28})$ must read the value of Q written by T_{28} , rather than the value that T_{27} is attempting to write.

This observation leads to a modified version of the timestamp-ordering protocol in which obsolete `write` operations can be ignored under certain circumstances. The protocol rules for `read` operations remain unchanged. The protocol rules for `write` operations, however, are slightly different from the timestamp-ordering protocol of Section 18.5.2.

The modification to the timestamp-ordering protocol, called **Thomas' write rule**, is this: Suppose that transaction T_i issues `write( $Q$ )`.

1. If $TS(T_i) < R\text{-timestamp}(Q)$, then the value of Q that T_i is producing was previously needed, and it had been assumed that the value would never be produced. Hence, the system rejects the `write` operation and rolls T_i back.
2. If $TS(T_i) < W\text{-timestamp}(Q)$, then T_i is attempting to write an obsolete value of Q . Hence, this `write` operation can be ignored.
3. Otherwise, the system executes the `write` operation and sets $W\text{-timestamp}(Q)$ to $TS(T_i)$.

The difference between these rules and those of Section 18.5.2 lies in the second rule. The timestamp-ordering protocol requires that T_i be rolled back if T_i issues `write( $Q$ )` and $TS(T_i) < W\text{-timestamp}(Q)$. However, here, in those cases where $TS(T_i) \geq R\text{-timestamp}(Q)$, we ignore the obsolete `write`.

By ignoring the write, Thomas' write rule allows schedules that are not conflict serializable but are nevertheless correct. Those non-conflict-serializable schedules allowed satisfy the definition of *view serializable* schedules (see Note 18.1 on page 867). Thomas' write rule makes use of view serializability by, in effect, deleting obsolete `write` operations from the transactions that issue them. This modification of transactions makes it possible to generate serializable schedules that would not be possible under the other

protocols presented in this chapter. For example, schedule 4 of Figure 18.18 is not conflict serializable and, thus, is not possible under the two-phase locking protocol, the tree protocol, or the timestamp-ordering protocol. Under Thomas' write rule, the `write(Q)` operation of T_{27} would be ignored. The result is a schedule that is *view equivalent* to the serial schedule $\langle T_{27}, T_{28} \rangle$.

18.6 Validation-Based Protocols

In cases where a majority of transactions are read-only transactions, the rate of conflicts among transactions may be low. Thus, many of these transactions, if executed without the supervision of a concurrency-control scheme, would nevertheless leave the system in a consistent state. A concurrency-control scheme imposes overhead of code execution and possible delay of transactions. It may be better to use an alternative scheme that imposes less overhead. A difficulty in reducing the overhead is that we do not know in advance which transactions will be involved in a conflict. To gain that knowledge, we need a scheme for *monitoring* the system.

The **validation protocol** requires that each transaction T_i executes in two or three different phases in its lifetime, depending on whether it is a read-only or an update transaction. The phases are, in order:

1. **Read phase.** During this phase, the system executes transaction T_i . It reads the values of the various data items and stores them in variables local to T_i . It performs all `write` operations on temporary local variables, without updates of the actual database.
2. **Validation phase.** The validation test (described below) is applied to transaction T_i . This determines whether T_i is allowed to proceed to the write phase without causing a violation of serializability. If a transaction fails the validation test, the system aborts the transaction.
3. **Write phase.** If the validation test succeeds for transaction T_i , the temporary local variables that hold the results of any write operations performed by T_i are copied to the database. Read-only transactions omit this phase.

Each transaction must go through the phases in the order shown. However, phases of concurrently executing transactions can be interleaved.

To perform the validation test, we need to know when the various phases of transactions took place. We shall, therefore, associate three different timestamps with each transaction T_i :

1. **StartTS(T_i)**, the time when T_i started its execution.
2. **ValidationTS(T_i)**, the time when T_i finished its read phase and started its validation phase.
3. **FinishTS(T_i)**, the time when T_i finished its write phase.

Note 18.1 VIEW SERIALIZABILITY

There is another form of equivalence that is less stringent than conflict equivalence, but that, like conflict equivalence, is based on only the **read** and **write** operations of transactions.

Consider two schedules S and S' , where the same set of transactions participates in both schedules. The schedules S and S' are said to be **view equivalent** if three conditions are met:

1. For each data item Q , if transaction T_i reads the initial value of Q in schedule S , then transaction T_i must, in schedule S' , also read the initial value of Q .
2. For each data item Q , if transaction T_i executes $\text{read}(Q)$ in schedule S , and if that value was produced by a $\text{write}(Q)$ operation executed by transaction T_j , then the $\text{read}(Q)$ operation of transaction T_i must, in schedule S' , also read the value of Q that was produced by the same $\text{write}(Q)$ operation of transaction T_j .
3. For each data item Q , the transaction (if any) that performs the final $\text{write}(Q)$ operation in schedule S must perform the final $\text{write}(Q)$ in schedule S' .

Conditions 1 and 2 ensure that each transaction reads the same values in both schedules and, therefore, performs the same computation. Condition 3, coupled with conditions 1 and 2, ensures that both schedules result in the same final system state.

The concept of view equivalence leads to the concept of view serializability. We say that a schedule S is **view serializable** if it is view equivalent to a serial schedule.

As an illustration, suppose that we augment schedule 4 with transaction T_{29} and obtain the following view serializable (schedule 5):

T_{27}	T_{28}	T_{29}
read (Q)	write (Q)	
write (Q)		write (Q)

Indeed, schedule 5 is view equivalent to the serial schedule $\langle T_{27}, T_{28}, T_{29} \rangle$, since the one $\text{read}(Q)$ instruction reads the initial value of Q in both schedules and T_{29} performs the final write of Q in both schedules.

Every conflict-serializable schedule is also view serializable, but there are view-serializable schedules that are not conflict serializable. Indeed, schedule 5 is not conflict serializable, since every pair of consecutive instructions conflicts, and, thus, no swapping of instructions is possible.

Note 18.1 VIEW SERIALIZABILITY (Cont.)

Observe that, in schedule 5, transactions T_{28} and T_{29} perform $\text{write}(Q)$ operations without having performed a $\text{read}(Q)$ operation. Writes of this sort are called **blind writes**. Blind writes appear in any view-serializable schedule that is not conflict serializable.

We determine the serializability order by the timestamp-ordering technique, using the value of the timestamp $\text{ValidationTS}(T_i)$. Thus, the value $\text{TS}(T_i) = \text{ValidationTS}(T_i)$ and, if $\text{TS}(T_j) < \text{TS}(T_k)$, then any produced schedule must be equivalent to a serial schedule in which transaction T_j appears before transaction T_k .

The **validation test** for transaction T_i requires that, for all transactions T_k with $\text{TS}(T_k) < \text{TS}(T_i)$, one of the following two conditions must hold:

1. $\text{FinishTS}(T_k) < \text{StartTS}(T_i)$. Since T_k completes its execution before T_i started, the serializability order is indeed maintained.
2. The set of data items written by T_k does not intersect with the set of data items read by T_i , and T_k completes its write phase before T_i starts its validation phase ($\text{StartTS}(T_i) < \text{FinishTS}(T_k) < \text{ValidationTS}(T_i)$). This condition ensures that the writes of T_k and T_i do not overlap. Since the writes of T_k do not affect the read of T_i , and since T_i cannot affect the read of T_k , the serializability order is indeed maintained.

As an illustration, consider again transactions T_{25} and T_{26} . Suppose that $\text{TS}(T_{25}) < \text{TS}(T_{26})$. Then, the validation phase succeeds in the schedule 6 in Figure 18.19. Note that the writes to the actual variables are performed only after the validation phase of T_{26} . Thus, T_{25} reads the old values of B and A , and this schedule is serializable.

The validation scheme automatically guards against cascading rollbacks, since the actual writes take place only after the transaction issuing the write has committed. However, there is a possibility of starvation of long transactions, due to a sequence of conflicting short transactions that cause repeated restarts of the long transaction. To avoid starvation, conflicting transactions must be temporarily blocked to enable the long transaction to finish.

Note also that the validation conditions result in a transaction T only being validated against the set of transactions T_i that finished after T started, and, further, are serialized before T . Transactions that finished before T started can be ignored in the validation tests. Transactions T_i that are serialized after T (that is, they have $\text{ValidationTS}(T_i) > \text{ValidationTS}(T)$) can also be ignored; when such a transaction T_i is validated, it would be validated against T if T finished after T_i started.